About this PDF

Built only from: **Theory of Computation (Automata and Formal Languages)** — https://nptel.ac.in/courses/106104028

Licence: CC BY-SA 4.0. Accessed .

Covers 11 of 11 syllabus sub-topics of this unit.

Normal forms for CFG

Definition

A context-free grammar is in **Chomsky Normal Form (CNF)** if every production rule is of the form A -> B C or A -> a, where A, B, C are non-terminal variables and a is a terminal symbol. A grammar is in **Greibach Normal Form (GNF)** if every production rule is of the form A -> a alpha, where a is a terminal symbol and alpha is a string of zero or more non-terminal variables.

Explanation

Before converting a CFG into canonical normal forms, it must undergo simplification to eliminate redundant symbols and rules without altering the generated language. The simplification consists of three phases:

1. **Elimination of Useless Symbols** - identify generating symbols (variables that derive terminal strings) and discard non-generating symbols; then identify reachable symbols (symbols reachable from the start symbol S) and discard unreachable symbols.
2. **Elimination of Epsilon-Productions** - determine nullable variables (variables A such that A =>* epsilon); for every production containing nullable variables, introduce new production rules representing all non-empty combinations where nullable variables are absent.
3. **Elimination of Unit Productions** - replace unit productions of the form A -> B (where both A and B are variables) by adding productions A -> alpha whenever B =>* alpha and alpha is not a single variable. After simplification, a CFG generating a language without epsilon can be converted into an equivalent grammar in CNF. In CNF, any derivation of a string of length n requires exactly 2n − 1 derivation steps. GNF provides a direct alignment between grammar productions and real-time pushdown automata transitions.

Algorithm / Steps

1. **Identify and remove useless symbols**: a. Find generating symbols. b. Discard non-generating symbols. c. Find reachable symbols from the start symbol. d. Discard unreachable symbols.
2. **Eliminate epsilon-productions**: a. Determine nullable variables. b. For each production containing nullable variables, add new productions for all non-empty combinations where nullable variables are omitted.
3. **Eliminate unit productions**: a. For each unit production A -> B, add productions A -> alpha for every B =>* alpha where alpha is not a single variable.

4. **Convert to CNF**: a. Ensure every production is of the form A -> B C or A -> a.
5. **Convert to GNF**: a. Ensure every production is of the form A -> a alpha.

Key points

- Simplification removes useless symbols, epsilon-productions, and unit productions before normal form conversion.
- CNF requires productions to be either two non-terminals or a single terminal.
- GNF requires productions to start with a terminal followed by zero or more non-terminals.
- In CNF, a derivation of a string of length n uses exactly 2n − 1 steps.
- GNF aligns grammar productions with real-time pushdown automata transitions.

Simplification of CFG

Definition

Simplification of a context-free grammar removes symbols and rules that do not affect the language it generates.

Explanation

The process consists of three main eliminations: useless symbols, epsilon-productions, and unit productions. Each elimination preserves the language while reducing the grammar.

Algorithm / Steps

1. Identify generating symbols (variables that can derive terminal strings) and discard non-generating symbols.
2. Identify reachable symbols (symbols reachable from the start symbol S) and discard unreachable symbols.
3. Determine nullable variables (variables A such that A =>* epsilon). For every production containing nullable variables, introduce new production rules representing all non-empty combinations where nullable variables are absent.
4. Replace each unit production A -> B by adding productions A -> alpha for every derivation B =>* alpha where alpha is not a single variable.

Key points

- Useless symbols are removed in two phases: generating symbols first, then reachable symbols.
- Epsilon-productions are eliminated by generating new rules for all non-empty combinations of nullable variables.
- Unit productions are replaced by directly adding the productions of the target variable.

Chomsky Normal Form (CNF) and Greibach Normal Form (GNF)

Definition

- CNF: every production rule is of the form A -> B C or A -> a, where A, B, C are non-terminal variables and a is a terminal symbol.
- GNF: every production rule is of the form A -> a alpha, where a is a terminal symbol and alpha is a string of zero or more non-terminal variables.

Explanation

- A CFG must be simplified before converting to CNF or GNF to eliminate redundant symbols and rules without changing the language.
- Simplification involves eliminating useless symbols, epsilon-productions, and unit productions.
- In CNF, any derivation of a string of length n requires exactly 2n - 1 derivation steps.
- GNF aligns grammar productions with real-time pushdown automata transitions.

Algorithm / Steps

1. Identify generating symbols and discard non-generating symbols.
2. Identify reachable symbols and discard unreachable symbols.
3. Determine nullable variables. For every production containing nullable variables, introduce new production rules representing all non-empty combinations where nullable variables are absent.
4. Replace unit productions A -> B by adding productions A -> alpha whenever B =>* alpha where alpha is not a single variable.
5. After simplification, ensure each production is either A -> B C or A -> a for CNF, or A -> a alpha for GNF.

Key points

- CNF requires productions of the form A -> B C or A -> a.
- GNF requires productions of the form A -> a alpha.
- Simplification removes useless symbols, epsilon-productions, and unit productions.
- In CNF, derivations of length n use exactly 2n - 1 steps.
- GNF provides a direct correspondence with real-time pushdown automata transitions.

Pumping lemma for CFL

Definition

The pumping lemma for context-free languages states that for any context-free language L there exists a pumping length constant p such that any string z in L with |z| >= p can be written as z = u v w x y where the substrings satisfy: (1) |v x| >= 1 (at least one of v or x is non-empty), (2) |v w x| <= p, and (3) for all i >= 0 the string u v^i w x^i y is in L.

Explanation

The lemma is primarily used as an adversarial proof technique to establish that specific languages, such as L = { a^n b^n c^n | n >= 1 or L = { w w | w in {0,1^* , are not context-free.

Key points

- There exists a pumping length constant p for every context-free language.
- Any sufficiently long string in the language can be decomposed into five substrings u v w x y.
- At least one of the substrings v or x must be non-empty.
- The combined length of v w x is bounded by p.
- Pumping the substrings v and x any number of times (including zero) yields strings that remain in the language.

Closure properties of Context Free Languages

Definition

Context-free languages are closed under union, concatenation, and Kleene star operations. They are also closed under substitution, homomorphism, inverse homomorphism, and intersection with regular languages. However, they are not closed under intersection and complementation.

Explanation

The closure properties mean that applying these operations to languages in the class yields a language that remains in the class. For union, concatenation, Kleene star, substitution, homomorphism, inverse homomorphism, and intersection with regular languages, the resulting language is still context-free. In contrast, intersection of two CFLs can produce a language that is not context-free, and complementation of a CFL can also yield a language that is not context-free.

Key points

- Closed under union, concatenation, Kleene star.
- Closed under substitution, homomorphism, inverse homomorphism, intersection with regular languages.
- Not closed under intersection of two CFLs.
- Not closed under complementation.

Turing Machine : Basic model

Definition

A Turing Machine (TM) is a fundamental mathematical model of general-purpose computation introduced by Alan Turing in 1936. It models the capabilities of mechanical computing devices by manipulating symbols on an unbounded tape.

The basic Turing Machine model consists of three principal components:

- Finite State Control: A control unit that maintains the current internal state chosen from a finite set of states Q.
- Tape: An infinite one-dimensional tape divided into discrete cells, each capable of holding a single symbol from the tape alphabet Gamma or a special blank symbol B.
- Read/Write Head: A movable tape head that scans exactly one cell at any time, reads the symbol, writes a replacement symbol, and moves left (L) or right (R) by one position.

At each discrete computational step, based on the current state and scanned symbol, the machine transitions to a new state, overwrites the scanned symbol, and moves the head left or right.

definition and representation

Definition

A Turing machine is a mathematical model of computation consisting of a finite-state control, a read/write head, and an infinite one-dimensional tape divided into discrete cells, each holding a symbol from the tape alphabet or a blank symbol. A deterministic Turing machine is formally defined as a 7-tuple M = (Q, Sigma, Gamma, delta, q_0, B, F) where Q is a finite non-empty set of states, Sigma is the finite input alphabet (B not in Sigma), Gamma is the finite tape alphabet containing Sigma as a proper subset along with the blank symbol B, delta is the transition function mapping Q x Gamma to Q x Gamma x {L, R}, q_0 in Q is the initial start state, B in Gamma is the blank symbol, and F subset-of Q is the set of accepting states.

Explanation

The machine operates by reading the symbol under the head, writing a new symbol, and moving the head one cell left (L) or right (R) in each discrete step, according to the transition function delta.

A Turing machine can be formally represented through transition tables, state transition diagrams (directed graphs with labeled edges showing input/output/direction), or transition listings.

An Instantaneous Description represents the complete configuration of a TM at a given instant. It is written as alpha_1 q alpha_2, where q in Q is the current state, alpha_1 alpha_2 in Gamma^* represents the non-blank portion of the tape, and the read/write head is positioned currently scanning the first symbol of alpha_2. The turnstile |-- denotes a single-step move between configurations.

Algorithm / Steps

1. Write the current configuration as alpha_1 q alpha_2.
2. Use the transition function delta to determine the next state, symbol to write, and head movement.
3. Update the tape and head position to obtain the next ID, and denote the transition with |--.

Key points

- A Turing machine is a 7-tuple (Q, Sigma, Gamma, delta, q_0, B, F) defining its components.
- The machine reads, writes, and moves the head left or right in each step.
- Transition tables, diagrams, or listings are common representations of a TM.
- An Instantaneous Description alpha_1 q alpha_2 captures the entire configuration at a moment.
- The turnstile |-- indicates a single transition between IDs.

Instantaneous Description

Definition

An Instantaneous Description represents the complete configuration of a TM at a given instant. It is written as alpha1 q alpha2, where qinQ is the current state, alpha1alpha2inGamma* represents the non-blank portion of the tape, and the read/write head is positioned currently scanning the first symbol of alpha2.

Explanation

The turnstile |-- denotes a single-step move between configurations, indicating how the machine transitions from one ID to the next during computation. Language acceptance is defined in terms of IDs: a string w is accepted if starting from the initial configuration q0 w, the machine reaches an ID alpha1 p alpha2 where p is an accepting state, after a finite number of steps. If a string is not in the language, the machine may halt in a non-accepting state or loop indefinitely, reflecting the behavior of recursively enumerable languages.

Example

For a string w in L, the computation can be represented as q0 w |--* alpha1 p alpha2, where the asterisk indicates a sequence of one or more single-step moves between IDs, culminating in an accepting state p.

Key points

- An ID captures the entire machine configuration: tape contents, current state, and head position.
- The |-- symbol represents a single transition between IDs.
- Acceptance of a language is characterized by reaching an ID with an accepting state after finitely many steps.
- Non-accepting strings may cause the machine to halt in a non-accepting state or run forever.

Language acceptance by TM

Definition

A language L subset-of Sigma* is accepted by a Turing machine M if for every string w in L, the machine starting in configuration q0 w eventually reaches an accepting state in F and halts.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Explanation

If a string is not in L, the machine may halt in a non-accepting state or loop infinitely. Languages accepted by TMs are Recursively Enumerable (RE) languages; languages for which a TM halts on every input are Recursive languages (Decidable).

Algorithm / Steps

L(M) = { w in Sigma* | q0 w |--* alpha1 p alpha2 with p in F }

Key points

- Acceptance requires reaching an accepting state in F and halting.
- Non-accepted strings may cause the TM to halt non-acceptingly or loop forever.
- Languages accepted by TMs are Recursively Enumerable.
- Languages where the TM halts on every input are Recursive (Decidable).

TM as Computer of Integer functions

A Turing machine can compute integer functions by interpreting input strings as unary numbers and producing an output string on the tape.

Definition

TM as Computer of Integer Functions: Beyond language decision, a TM operates as an algorithm calculating mathematical functions f: N^k -> N.

Explanation

Non-negative integers are represented in unary notation, where an integer n is encoded as a string 0^n or 1^(n+1). Starting with input representations separated by blanks, the TM halts with the computed output value left on the tape. Functions computable by a TM are called Turing-computable or partial recursive functions.

Example

For example, a TM can compute integer addition, subtraction, multiplication, or comparison.

Algorithm / Steps

1. **Storage in the finite control**: Use composite state tuples [q, a, b] to remember a finite number of symbols within internal states.
2. **Multiple tracks**: Divide each tape cell into multiple fields to store data, markers, and scratch work simultaneously.
3. **Subroutines**: Modularize TM execution by writing subroutines (such as copy, shift, compare, or search routines) where a main routine transitions to a subroutine initial state and returns to a designated return state upon completion.

Key points

- A TM can compute functions f: N^k -> N using unary input representation.
- The machine halts with the output value left on the tape.

- Computable functions are called Turing-computable or partial recursive.
- Programming techniques include storage in finite control, multiple tracks, and subroutines.

Programming techniques for Turing machines (subroutines).

Definition

Subroutines are modularized parts of a Turing machine that can be invoked by the main routine and return control after completion.

Explanation

They work by having the main routine transition to a subroutine's initial state, execute the subroutine, and then return to a designated return state.

Example

Examples of subroutines include copy, shift, compare, and search routines.

Algorithm / Steps

1. The main routine transitions to the subroutine's initial state.
2. The subroutine executes its operations.
3. The subroutine returns to the designated return state.

Key points

- Subroutines modularize Turing machine execution.
- The main routine invokes a subroutine by transitioning to its initial state.
- After execution, the subroutine returns control to a designated return state.
- Typical subroutines include copy, shift, compare, and search routines.

Source and licence

This PDF is adapted from **Theory of Computation (Automata and Formal Languages)** (https://nptel.ac.in/courses/106104028), accessed .

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).